

Signal Processing, HPC/GPUs, and Interactive Plotting with MATLAB

CME 192 LECTURE 8

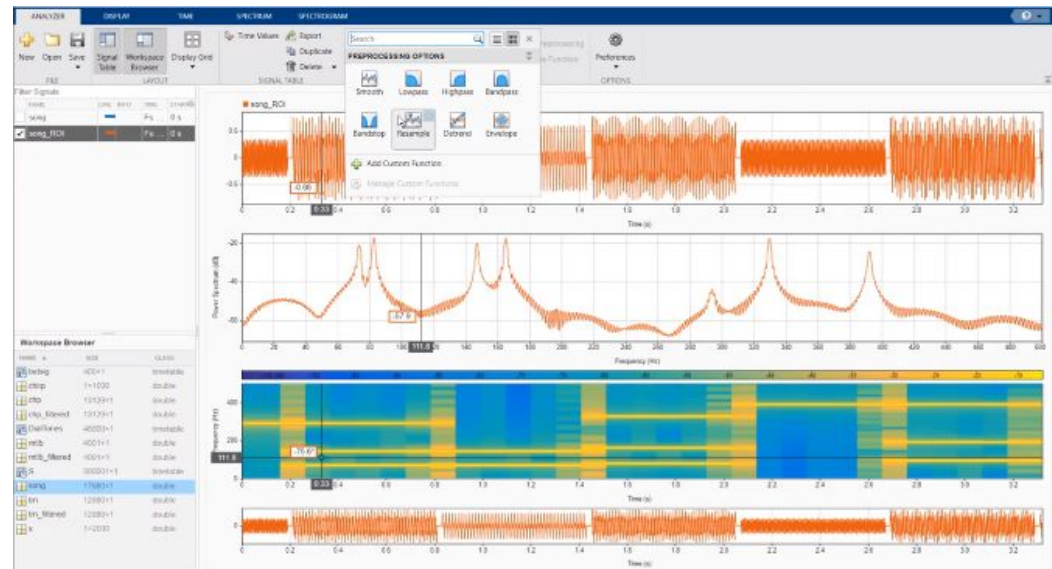
02/26/2026

Outline

- Signal Processing Toolbox
- Audio Toolbox
- DSP System Toolbox
- Parallel Processing and GPU
- Interactive Plotting

Signal Processing

Working with Signals



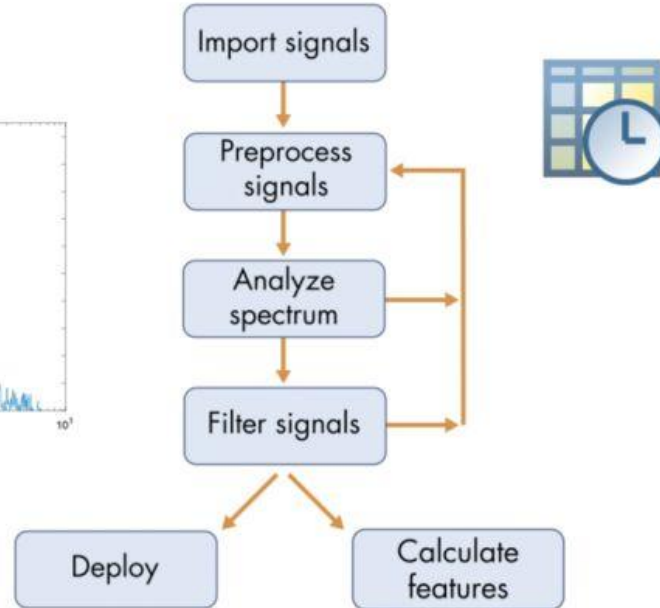
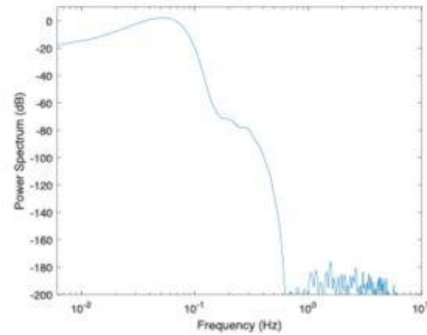
Signal Processing Toolbox

Analyze, preprocess, and extract features from uniformly and nonuniformly sampled signals

It provides tools and algorithms for:

- filter design and analysis
- resampling, smoothing, detrending
- power spectrum estimation
- extracting features like changepoints and envelopes
- finding peaks and signal patterns
- quantifying signal similarities
- performing measurements such as SNR and distortion
-

Signal Processing Pipeline



Some basic preprocessing steps:

- Resampling
 - `y = resample(x, p, q)`
 - resample the sequence `x` at p/q times the original sample rate
 - The length of the result `y` is p/q times the length of `x`.
- Normalizing
 - `normalize`
- Aligning
 - `synchronize`
 - `finddelay` (estimate the delay between signals)

Spectral Analysis

Any physical signal can be decomposed into a number of discrete frequencies, or a spectrum of frequencies over a continuous range.

Power Spectrum

- For a given signal, the power spectrum gives a plot of the portion of a signal's power (energy per unit time) falling within given frequency bins.
- `P = pspectrum(X)` returns the power spectrum of `X`.
- `pspectrum(...)` with no output arguments plots the spectral estimates.
- A **dB scale** is usually used to visualize power spectrums by calculating $10 \cdot \log_{10}(p)$, where p is the spectrum.

Spectrogram

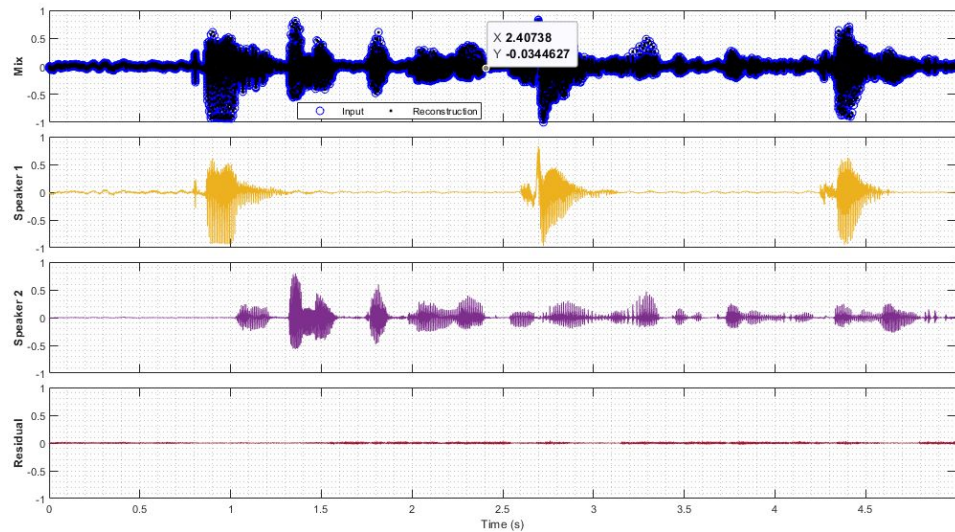
- `s = spectrogram(x)` returns the Short-Time Fourier Transform (STFT) of the input signal `x`. Each column of `s` contains an estimate of the short-term, time-localized frequency content of `x`.
- `spectrogram(____)` with no output arguments plots `ps` in decibels.

Filtering

Remove some frequencies or frequency bands from a signal.

- Lowpass filter
 - Low frequencies are passed, high frequencies are attenuated.
 - `y = lowpass(x, wpass)` filters the input signal `x` using a lowpass filter with normalized passband frequency `wpass` in units of π rad/sample.
- Highpass filter
 - High frequencies are passed, low frequencies are attenuated.
 - `y = highpass(x, wpass)` filters the input signal `x` using a highpass filter with normalized passband frequency `wpass` in units of π rad/sample.
- Bandpass filter
 - Only frequencies in a frequency band are passed.
 - `y = bandpass(x, wpass)` filters the input signal `x` using a bandpass filter with a passband frequency range specified by the two-element vector `wpass` and expressed in normalized units of π rad/sample.

Audio Toolbox



Audio Toolbox

It provides tools and algorithms for:

- processing audio signals such as equalization and time stretching
- estimating acoustic signal metrics such as loudness and sharpness
- extracting audio features such as MFCC and pitch
- advanced machine learning models, including i-vectors
- pretrained deep learning networks, including VGGish and CREPE
- live algorithm testing
- impulse response measurement
- signal labeling
-

- With Audio Toolbox you can import, label, and augment audio data sets, as well as extract features to train machine learning and deep learning models. The pre-trained models provided can be applied to audio recordings for high-level semantic analysis.
- You can prototype audio processing algorithms in real time or run custom acoustic measurements by streaming low-latency audio to and from sound cards. You can validate your algorithm by turning it into an audio plugin to run in external host applications such as Digital Audio Workstations.

Read and Write Audio Files

Call `audioread` with a file name to read the entire audio file and the sample rate of the audio.

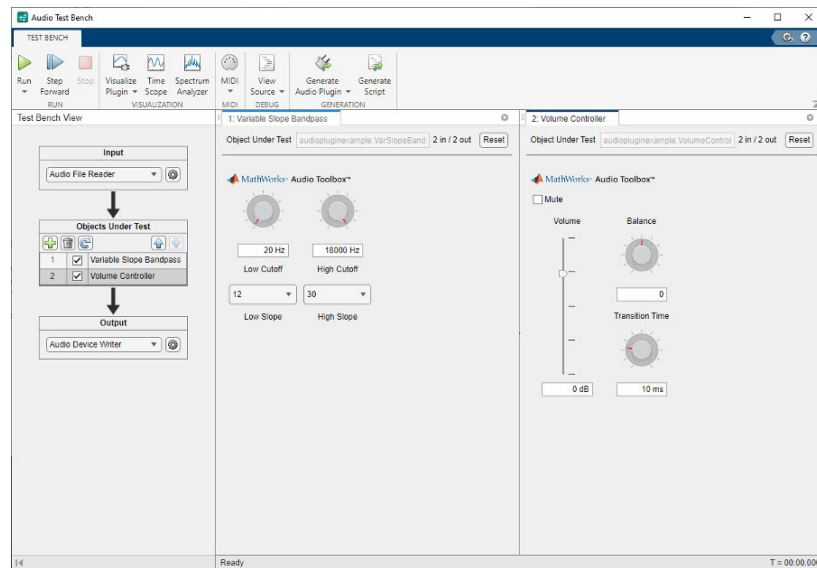
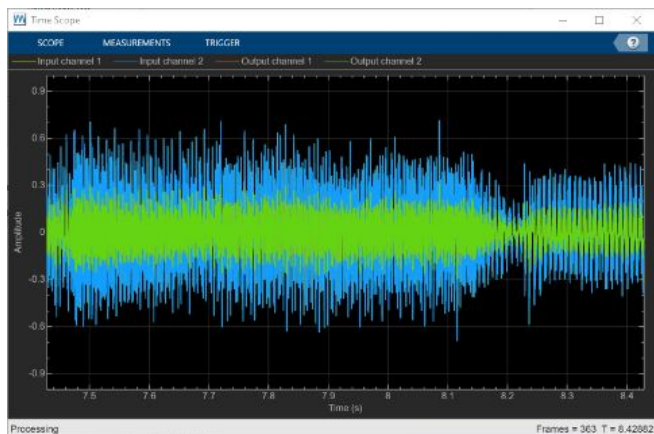
Call `soundsc` with the audio data and sample rate to play the audio to your default speakers.

Call `audiowrite` with the file name, the audio data, and the sample rate to write the audio to a file.

Audio Test Bench

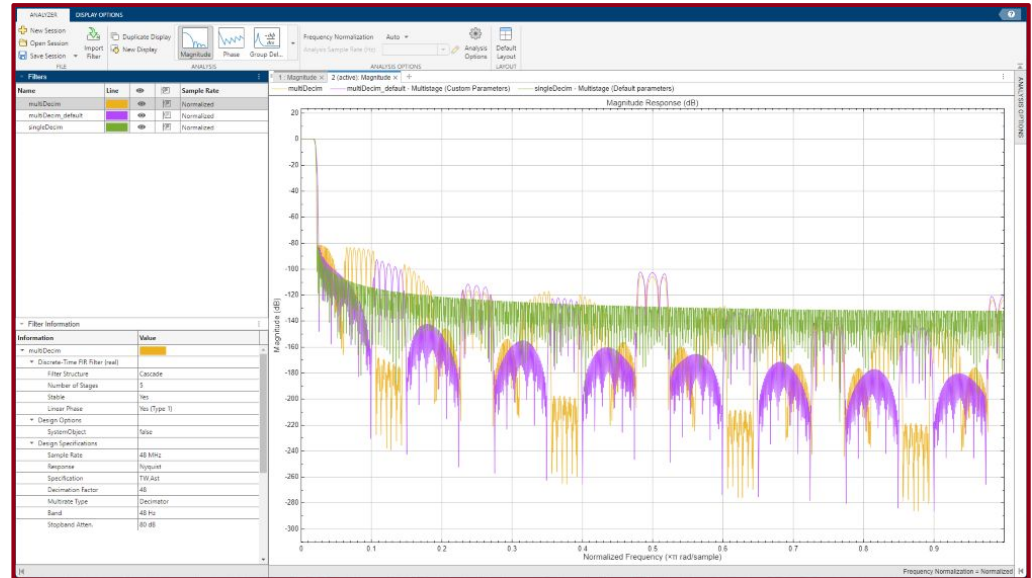
The Audio Test Bench app allows graphically setting up the audio input and output, processing audio, and opening common analysis tools like timescope and spectrumAnalyzer.

audioTestBench



Analyze the audio signal in the time and frequency domains

DSP System



DSP System Toolbox

Design and simulate streaming signal processing systems

It provides tools and algorithms for:

- design and analyze FIR, IIR, multirate, multistage, and adaptive filters
- stream signals from variables, data files, and network devices for system development and verification
- dynamically visualize and measure streaming signals using the Time Scope, Spectrum Analyzer, and Logic Analyzer
- support C/C++ code generation for desktop prototyping and deployment to embedded processors
- support bit-accurate fixed-point modeling and HDL code generation from filters, FFT, IFFT, and other algorithm
-

Visualize signals in frequency domain

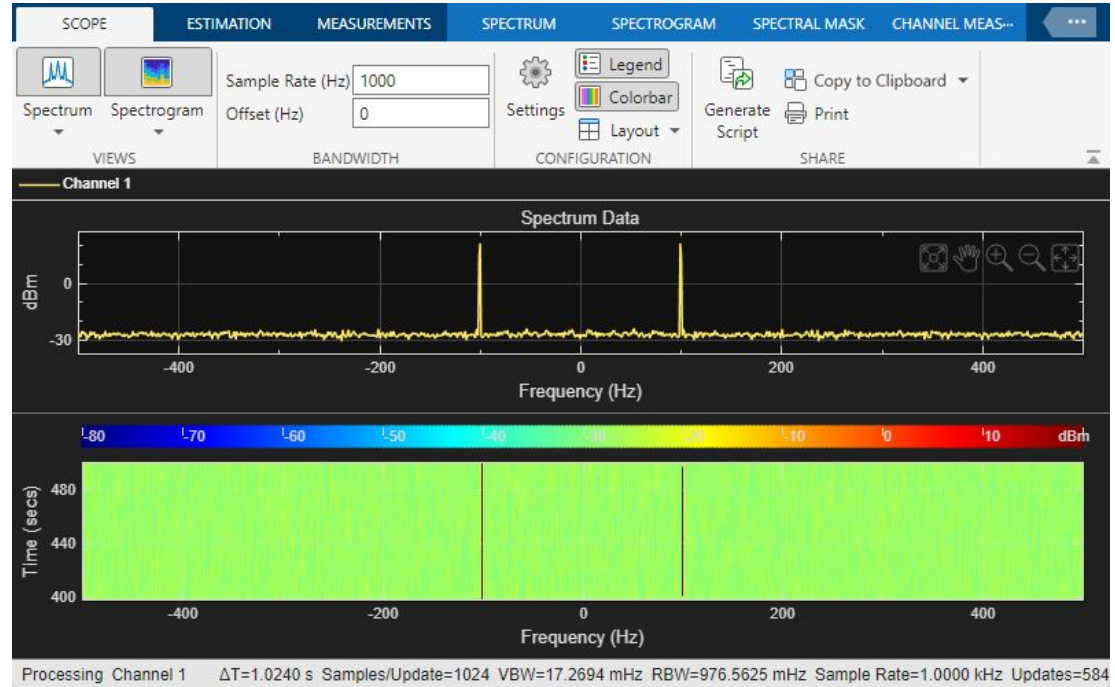
The **Spectrum Analyzer** object displays the frequency spectrum of time-domain signals.

Frame size is the first dimension of the input vector. The number of input channels must remain constant.

- Create the `dsp.SpectrumAnalyzer` object and call the object with arguments to display the spectra of signals in the Spectrum Analyzer.
- Set the `NumInputPorts` property to display multiple signals.
- `scope(signal1, signal2, ...)` displays multiple signals in the spectrum analyzer. The signals must have the same frame length, but can vary in number of channels.
- Release system resources of a System object named `obj` by `release(obj)`.

Spectrum Analyzer

- The spectrumAnalyzer object displays frequency-domain signals and the frequency spectrum of time-domain signals.
- The scope shows the spectrum view and the spectrogram view. The object performs spectral estimation using the filter bank method and Welch's method of averaged modified periodograms.
- You can customize the spectrum analyzer display to show the data and the measurement information that you need



Demo: extract voices from music signal

Implement a basic digital music synthesizer and use it to play a traditional song in a three-voice arrangement.

Signal Analyzer

MATLAB R2025b

HOME PLOTS APPS LIVE EDITOR INSERT VIEW

Design App Get More Apps Install App

Curve Fitter Optimization PID Tuner System Identification Wireless Waveform... Signal Analyzer Wireless Channel ... Instrument Control Simbiology Model Builder Simbiology Model ... MATLAB Coder Standalone Applicatio...

FILE

/ > Users > winnicki > Documents > MATLAB > Lecture_08 >

Files

Name

- data
- img_gs.jpg
- Lecture_08.mlx

Lecture_08.mlx x

A:\Users\winnicki\Documents\MATLAB\Lecture_08\Lecture_08.mlx

steepness to 0.95. Click **Apply**.

2. Select the medium signal by clicking its name in the Signal table. Select **Bandpass** from the **Functions** gallery. In the **Function Parameter** frequencies, respectively. Increase the steepness to 0.95. Click **Apply**.

3. Select the low signal by clicking its name in the Signal table. Select **Lowpass** from the **Functions** gallery. In the **Function Parameter** steepness to 0.95. Click **Apply**.

Select the three filtered signals by clicking their **Name** column in the Signal table.

On the **Analyzer** tab, click **Export** and save the signals to the current workspace.

Plot the spectra of the three signals.

```

118 % plot the spectra after exporting to current workspace
119 pspectrum(Low)
120 hold on
121 pspectrum(Medium)
122 pspectrum(High)
123 hold off

```

Signal Analyzer

ANALYZER DISPLAY TAB MEASUREMENTS

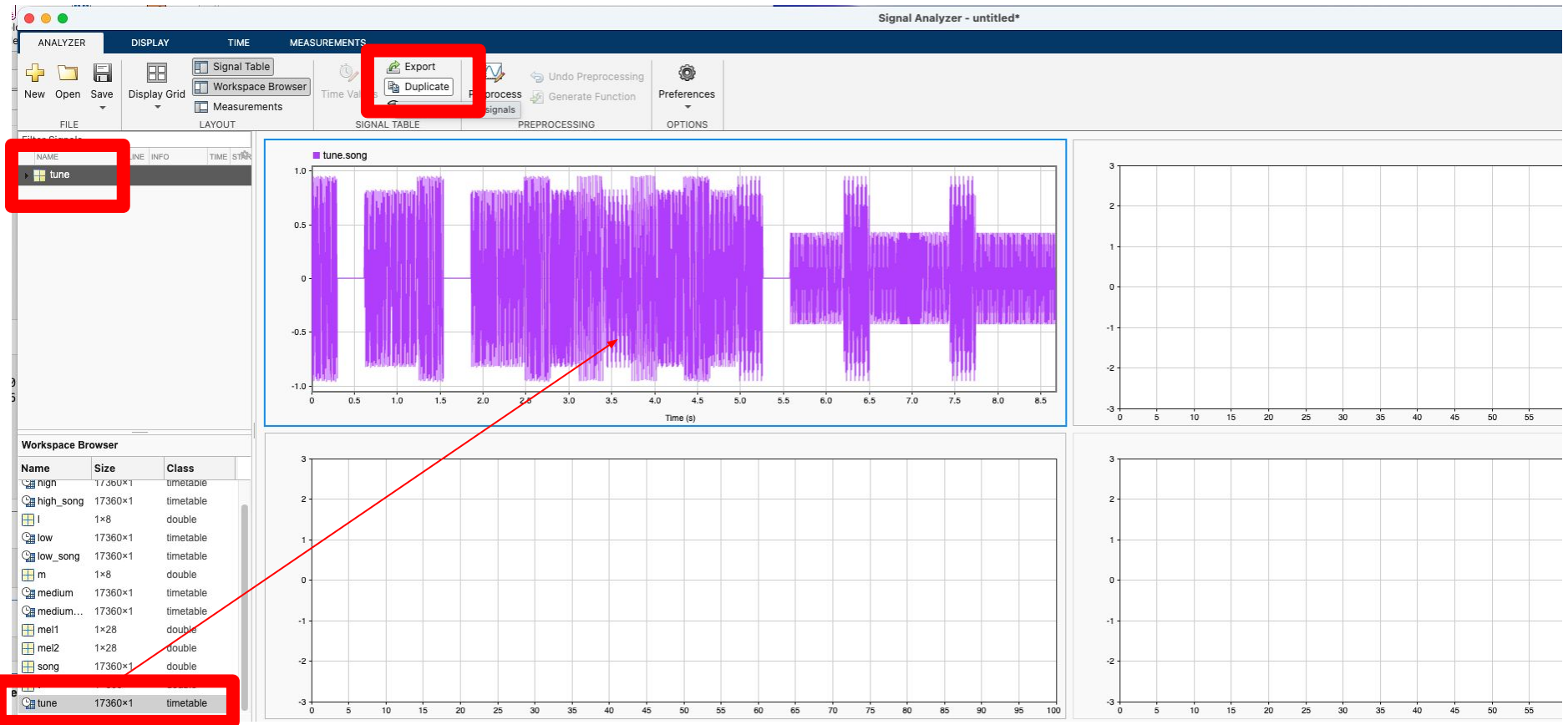
New Open Save Display Grid Workspace Browser Time Values Report Undo Preprocessing Generate Function Preferences

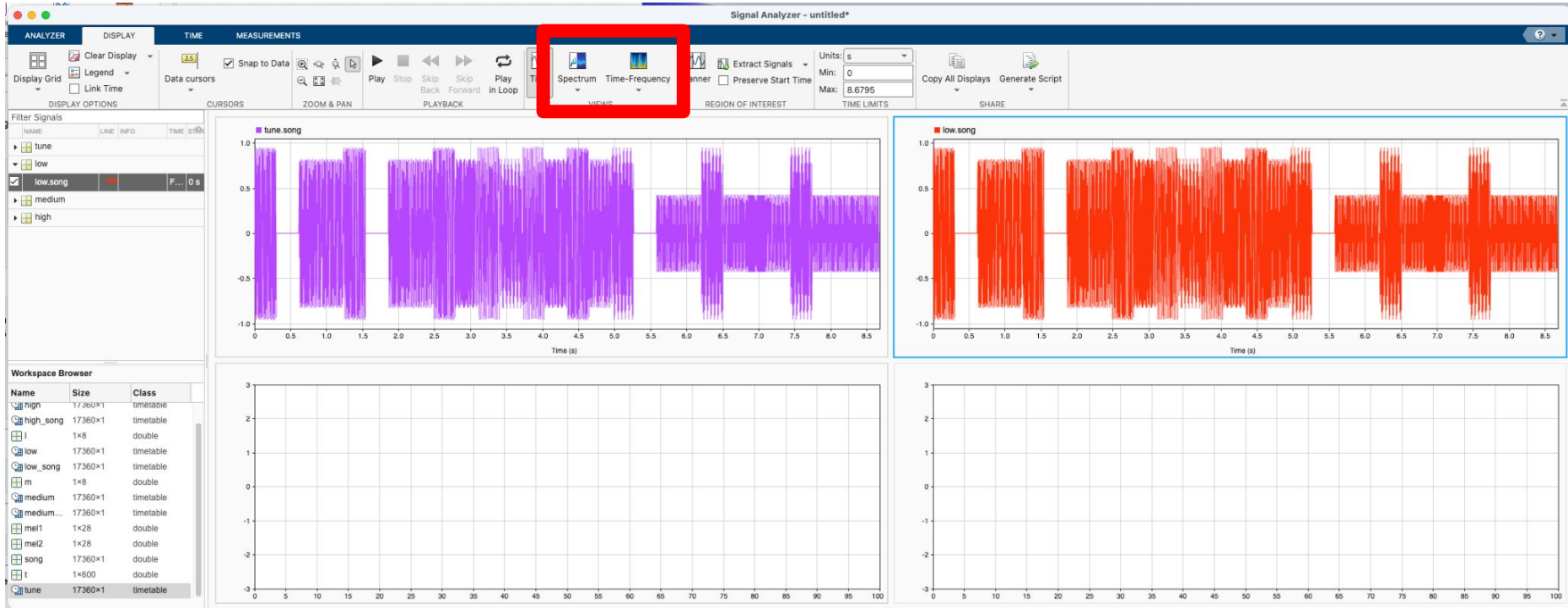
FILE LAYOUT SIGNAL TABLE PREPROCESSING OPTIONS

Time Signals Load Data Time Labels

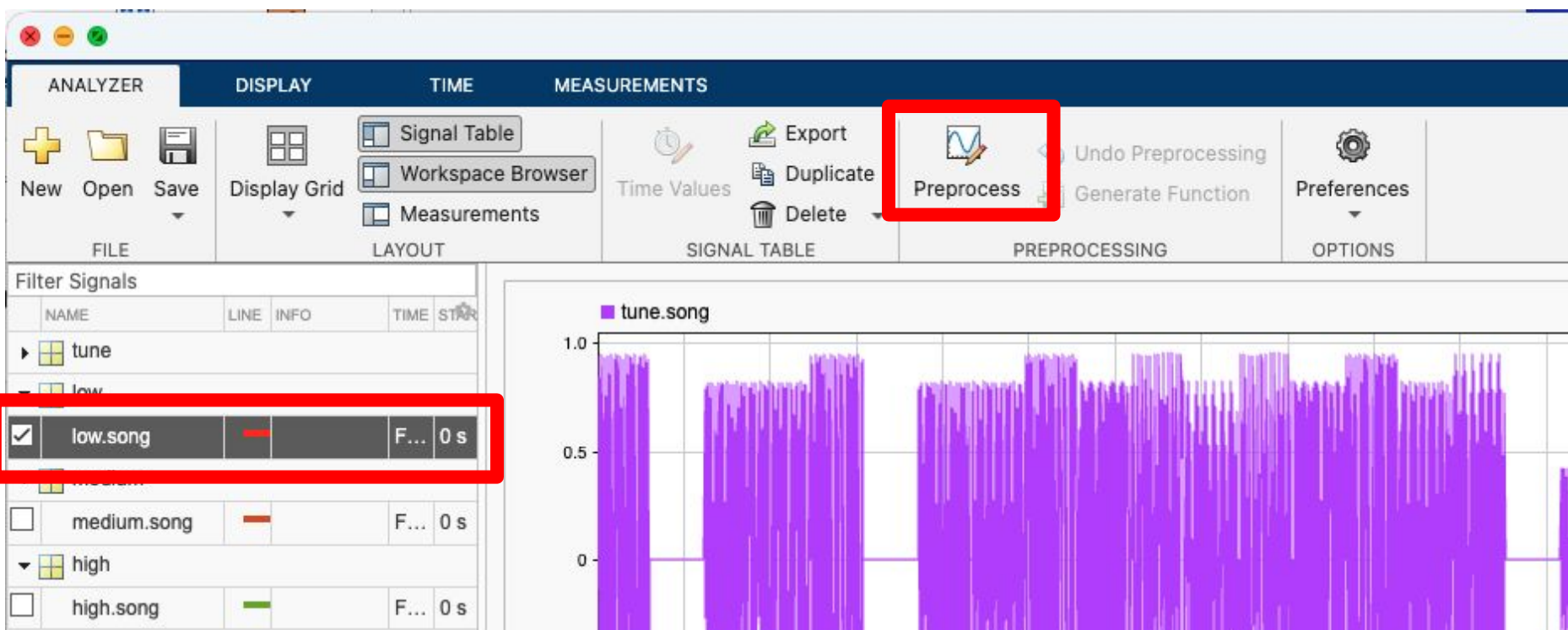
Workspace Browser

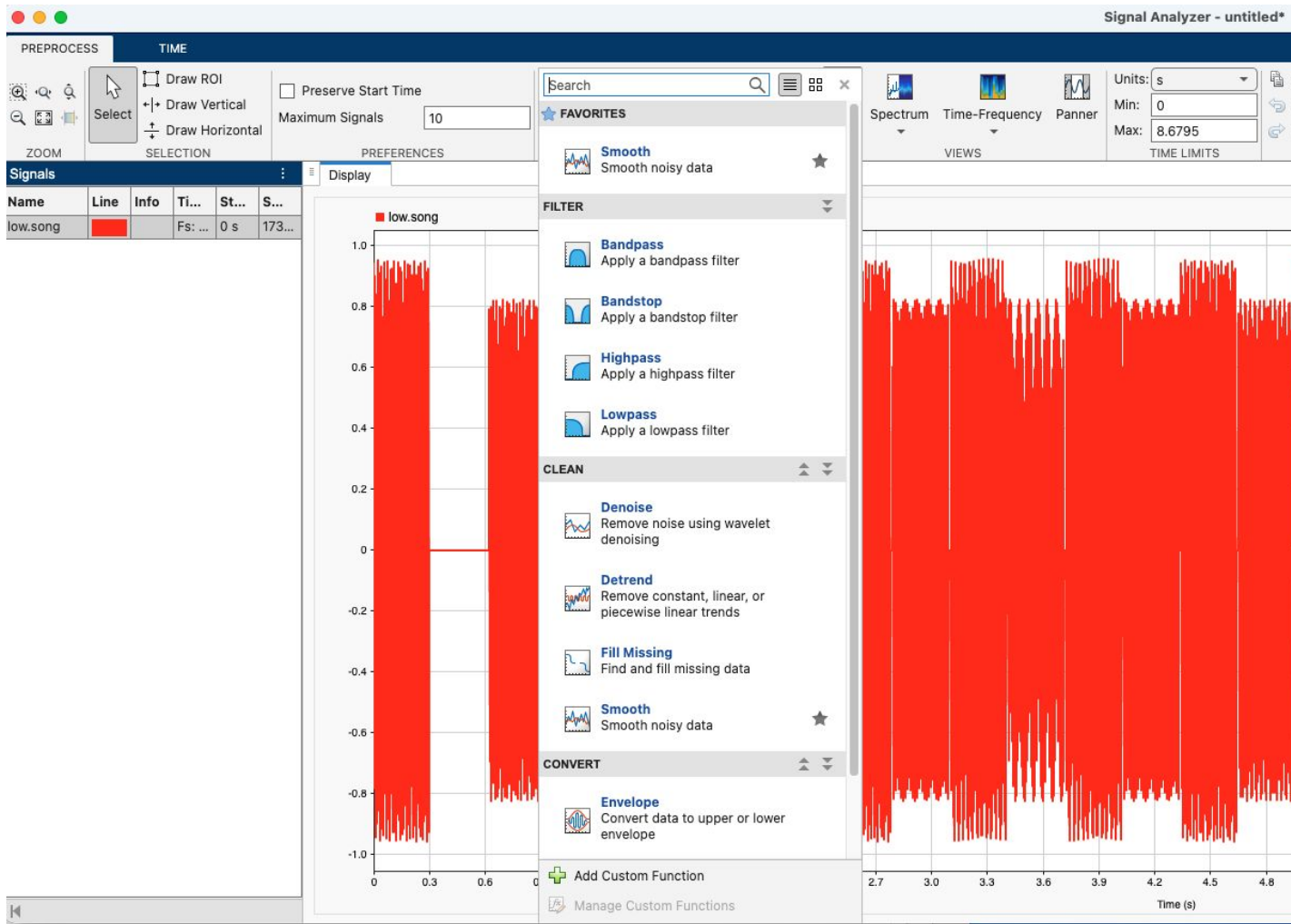
Name	Size	Class
Signal	17300x1	double
high_sing	17300x1	double
low	17300x1	double
low_sing	17300x1	double
med	17300x1	double
medium	17300x1	double
med2	17300x1	double
song	17300x1	double
tune	17300x1	double





Result after duplicating and renaming to low, medium, high.
Inspect the spectrum and time-frequency by clicking them.





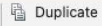


Units: s

Min: 0

Max: 8.6795

TIME LIMITS



Duplicate

Undo Preprocessing

Redo Preprocessing

SIGNAL

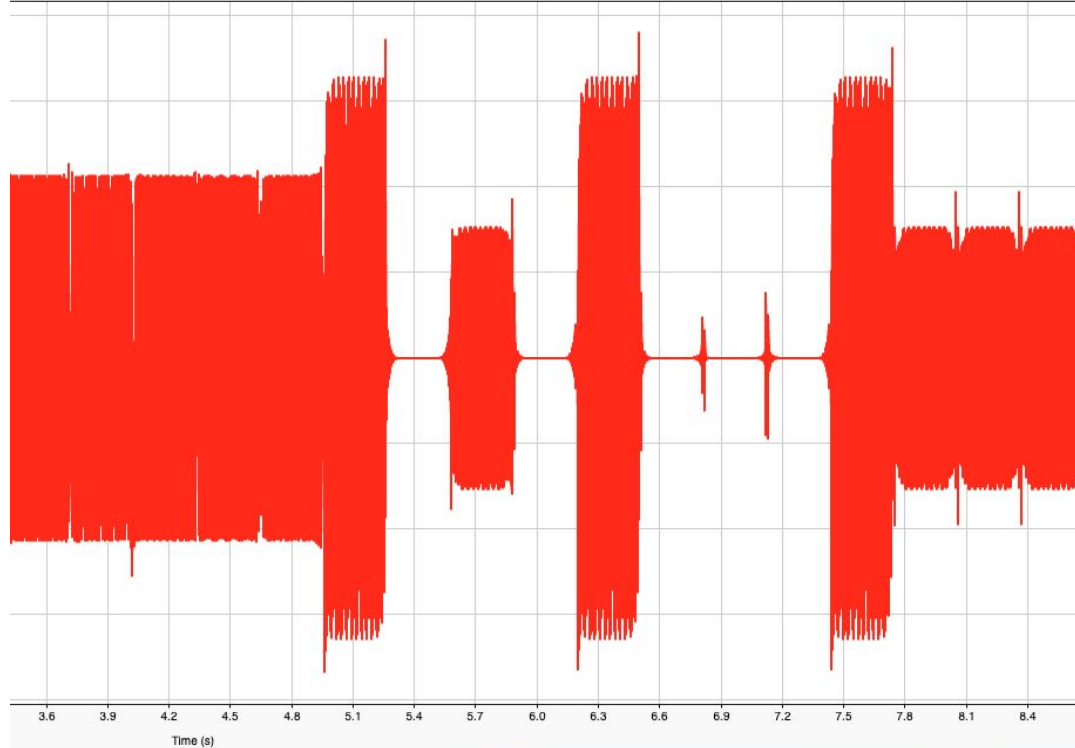


Accept

Cancel

All

CLOSE



Function Parameters

Highpass Parameters

▼ FILTER SPECIFICATIONS

Frequency Units: Hz

Passband Frequency: 450

Steepness: 0.95

Stopband Attenuation(dB): 60

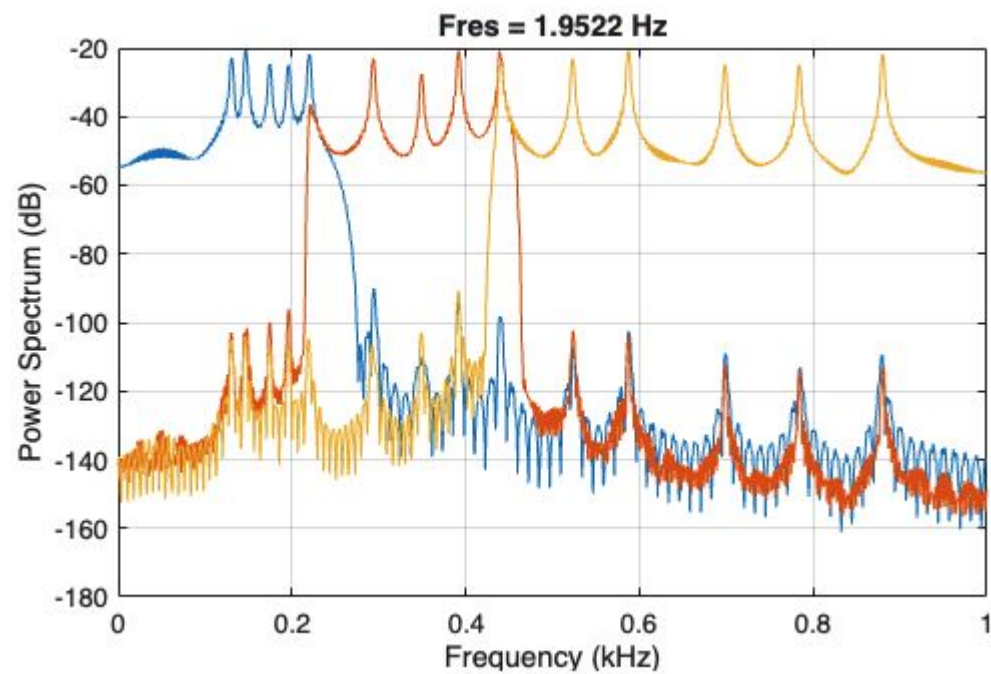
Help

Restore Defaults

Apply



Result after applying low, band, and high-pass filter



Parallel Computing and GPU Programming

Jump into HPC with MATLAB



MATLAB
Parallel Computing Toolbox

```
>> d = distributed(...);
```

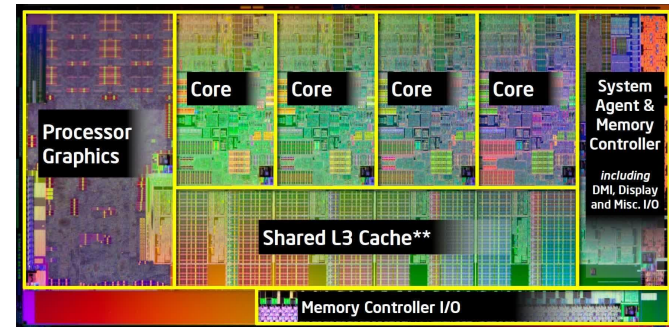
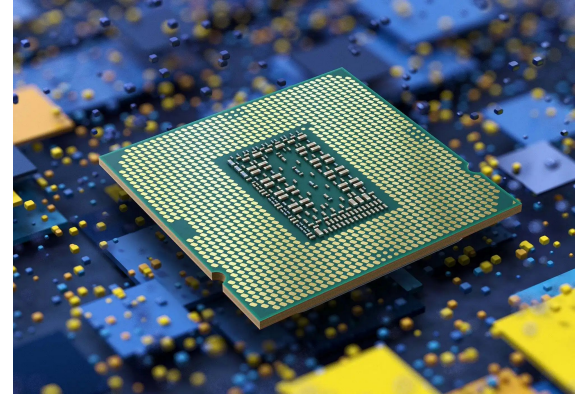


11	26	41	15	30	45	20	35	50	
12	27	42	16	31	46	...	21	36	51
13	28	43	17	32	47		22	37	52

MATLAB Parallel Server

Multicore Processors

- 4 to 64 cores
- Few but powerful. Optimized for both sequential and parallel computing.
- **General-purpose:** designed for a wide range of tasks with strong branch prediction, out-of-order execution, and large caches.
- **Flexible:** can handle diverse workloads including complex sequential tasks and multitasking
- **Characteristics:** high single-thread performance (e.g., silicon devoted to out of order execution, deeper pipelines, more superscalar execution units, and larger, more general caches) and shared memory.



GPUs, TPUs, and other specialized processing units

- A lot of cores but not as powerful.
- Often called accelerators.
- Specialized for parallelism: optimized for tasks that can be broken into many small, independent operations (e.g., graphics rendering, deep learning).
- Throughput-oriented: prioritize overall throughput over individual core performance, with less emphasis on branch prediction and cache hierarchy.
- Not suitable for sequential computations.
- Examples: NVIDIA and AMD GPUs, TPU (Tensor Processing Units).

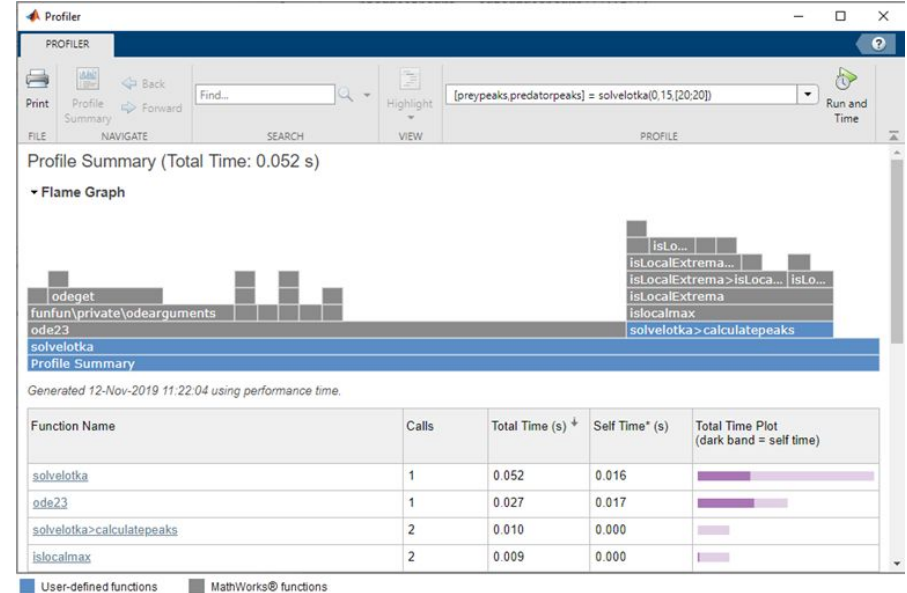
Best suited for...

- CPUs:
 - The general-purpose processor that runs the operating system and most application logic, designed for flexibility and complex decision-making. It has a small number of powerful cores optimized for sequential and branching tasks.
 - CPUs are best for workloads with irregular control flow, small-to-medium computations, or tasks that cannot be easily parallelized across many units.
 - Compiling, running a web server, database, query processing, irregular computations
- GPUs:
 - A highly parallel processor built to perform the same operation on many data elements simultaneously, originally for graphics rendering but now widely used in scientific computing and AI.
 - GPUs are ideal for large-scale numerical operations and other data-parallel workloads.
 - Matrix multiplication, physics simulation (on a grid), training deep neural networks, image and video processing, rendering 3D graphics
- Difference: CPUs prioritize low-latency execution and control logic with few strong cores, while GPUs prioritize high-throughput parallel computation with many lightweight cores.

Profiling Your Code

To profile your code and improve its performance, use this general process:

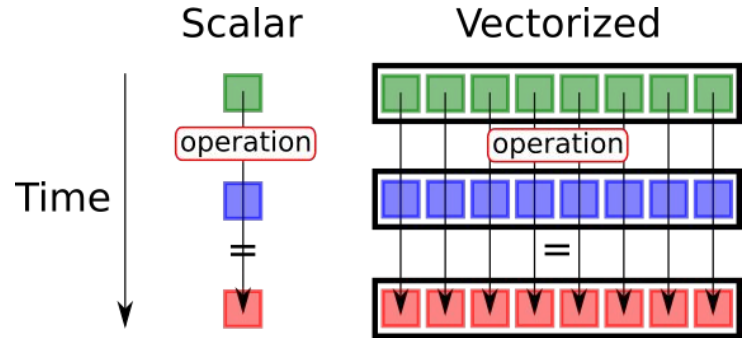
1. Run the Profiler on your code.
2. Review the profile summary results.
3. Investigate functions and individual lines of code.
For example, you may want to investigate functions and lines of code that use a significant amount of time or that are called most frequently.
4. Save the profiling results.
5. Implement potential performance improvements in your code.
For example, if you have a `load` statement within a loop, you might be able to move the `load` statement outside the loop so that it is called only once.
6. Save the files, and run `clear all`. Run the Profiler again and compare the results to the original results.
7. Repeat the above steps to continue improving the performance of your code. When your code spends most of its time on calls to a few built-in functions, you have probably optimized the code as much as possible.



Vectorization

MATLAB is optimized for operations involving matrices and vectors. The process of revising loop-based, scalar-oriented code to use MATLAB matrix and vector operations is called *vectorization*. Vectorizing your code is worthwhile for several reasons:

- *Appearance*: Vectorized mathematical code appears more like the mathematical expressions found in textbooks, making the code easier to understand.
- *Less Error Prone*: Without loops, vectorized code is often shorter. Fewer lines of code mean fewer opportunities to introduce programming errors.
- *Performance*: Vectorized code often runs much faster than the corresponding code containing loops.

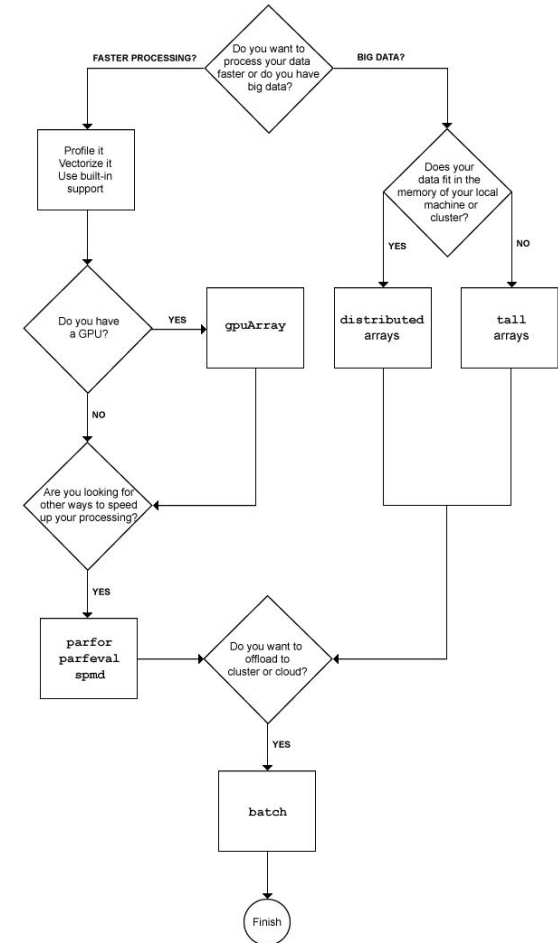


Vectorization Examples

- Replace explicit loops with array operations: Apply functions and arithmetic directly to whole vectors or matrices (e.g., $y = \sin(t)$ instead of looping), reducing interpreter overhead and leveraging optimized internal libraries.
- Use element-wise operators ($.*$, $./$, $.^$): Perform per-element computations across arrays without indexing through elements manually.
- Exploit implicit expansion (broadcasting): Write expressions like $A - \text{mean}(A)$ instead of looping over columns, letting MATLAB automatically expand size-1 dimensions.
- Use logical indexing instead of conditional loops: Select or modify subsets of data with masks (e.g., $x = x(x \geq 0)$), avoiding if statements inside loops.
- Preallocate arrays before filling them: Create arrays with zeros, ones, or nan before assignment to avoid costly dynamic resizing inside loops.
- Prefer built-in functions over custom implementations: Use optimized functions like `sum`, `cumsum`, `diff`, `unique`, and `sort` instead of manually coding the same behavior.

Choose a Parallel Computing Solution

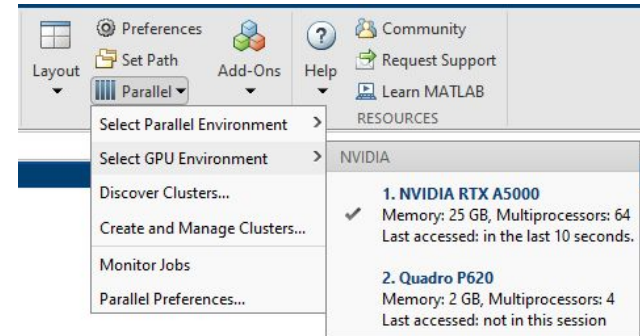
- Several MATLAB and Simulink products have a growing number of functions and features that help you take advantage of parallel computing resources without requiring any extra coding.
 - Run MATLAB Functions with Automatic Parallel Support:
UseParallel="auto"
- Interactively run loops in parallel using parfor
 - One key requirement for using parfor-loops is that the individual iterations must be independent. Independent problems suitable for parfor processing include Monte Carlo simulations and parameter sweeps.
- Run simulations in parallel using parsim
- Parfeval schedules the function to run in the background. You can run other code while MATLAB is running the function.
- For single program multiple dataset, use the smpd statement



Problem	Solutions	Required Products	More Information
Do you want to process your data faster?	Profile your code.	MATLAB	Profile Your Code to Improve Performance
	Vectorize your code.	MATLAB	Vectorization
	Use automatic parallel computing support in MathWorks® products.	MATLAB Parallel Computing Toolbox	Run MATLAB Functions with Automatic Parallel Support
	If you have a GPU, try <code>gpuArray</code> .	MATLAB, Parallel Computing Toolbox	Run MATLAB Functions on a GPU
	Use <code>parfor</code> .	MATLAB, Parallel Computing Toolbox	Interactively Run Loops in Parallel Using parfor
Do you want to speed up simulations?	Use <code>parsim</code> .	Simulink®, Parallel Computing Toolbox	Running Multiple Simulations (Simulink)
Are you looking for other ways to speed up your processing?	Try <code>parfeval</code> .	MATLAB, Parallel Computing Toolbox	Evaluate Functions in the Background Using parfeval
	Try <code>spmd</code> .	MATLAB, Parallel Computing Toolbox	Run Single Programs on Multiple Data Sets
Do you want to scale up your big data calculation?	To work with out-of-memory data with any number of rows, use tall arrays. This workflow is well suited to data analytics and machine learning.	MATLAB	Big Data Workflow Using Tall Arrays and Datastores
	Use tall arrays in parallel on your local machine.	MATLAB, Parallel Computing Toolbox	Use Tall Arrays on a Parallel Pool
	Use tall arrays in parallel on your cluster.	MATLAB, Parallel Computing Toolbox, MATLAB Parallel Server	Use Tall Arrays on a Spark Cluster
	If your data is large in multiple dimensions, use <code>distributed</code> instead. This workflow is well suited to linear algebra problems.	MATLAB, Parallel Computing Toolbox, MATLAB Parallel Server	Run MATLAB Functions with Distributed Arrays
Do you want to offload to a cluster?	Use <code>batch</code> to run your code on clusters and clouds.	MATLAB Parallel Server	Run Batch Parallel Jobs
	Use <code>batchsim</code> to run your simulations on clusters and clouds.	Simulink, MATLAB Parallel Server	Running Multiple Simulations (Simulink)

Run MATLAB Functions on a GPU

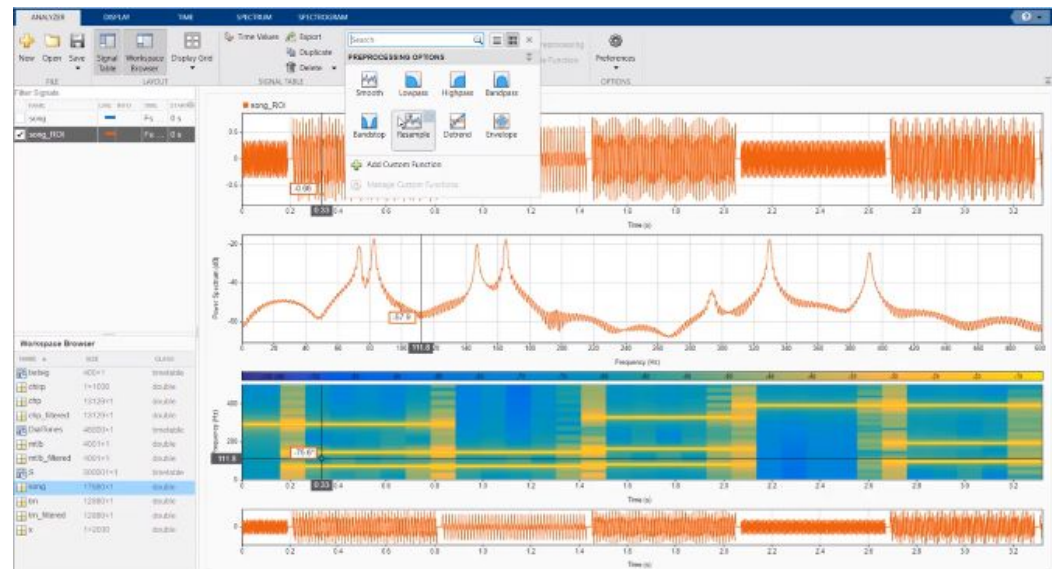
- Many functions in MATLAB and other toolboxes run automatically on a GPU if you supply a gpuArray data argument. A gpuArray in MATLAB represents an array that is stored on the GPU.
 - [gpuArray-enabled functions include the discrete Fourier transform \(fft\), matrix multiplication \(mtimes\), left matrix division \(mldivide\), and hundreds of others.](#)
- Whenever you call any of these functions with at least one gpuArray as a data input argument, the function executes on the GPU. The function generates a gpuArray as the result, unless returning numeric data to the local workspace is more appropriate (for example, size).
 - You can mix inputs using both gpuArray data and arrays stored in host memory in the same function call.
- If you have a supported GPU, then MATLAB automatically uses it for GPU computation. If you have multiple GPUs, then you can use gpuDeviceTable to examine the properties of all GPUs detected in your system. You can use gpuDevice to select one of them, or use multiple GPUs with a parallel pool.



Toolbox Name	List of Functions with <code>gpuArray</code> Support	GPU-Specific Documentation
MATLAB	Functions with <code>gpuArray</code> support	
Statistics and Machine Learning Toolbox™	Functions with <code>gpuArray</code> support (Statistics and Machine Learning Toolbox)	Analyze and Model Data on GPU (Statistics and Machine Learning Toolbox)
Image Processing Toolbox™	Functions with <code>gpuArray</code> support (Image Processing Toolbox)	GPU Computing (Image Processing Toolbox)
Medical Imaging Toolbox™	Functions with <code>gpuArray</code> support (Medical Imaging Toolbox)	
Deep Learning Toolbox™	Functions with <code>gpuArray</code> support (Deep Learning Toolbox) See also Deep Learning with GPUs	Scale Up Deep Learning in Parallel, on GPUs, and in the Cloud (Deep Learning Toolbox) Deep Learning with MATLAB on Multiple GPUs (Deep Learning Toolbox)
Computer Vision Toolbox™	Functions with <code>gpuArray</code> support (Computer Vision Toolbox)	GPU Code Generation and Acceleration (Computer Vision Toolbox)

Toolbox Name	List of Functions with <code>gpuArray</code> Support	GPU-Specific Documentation
Communications Toolbox™	Functions with <code>gpuArray</code> support (Communications Toolbox)	Acceleration (Communications Toolbox)
5G Toolbox™	Functions with <code>gpuArray</code> support (5G Toolbox)	
Antenna Toolbox™	Functions with <code>gpuArray</code> Support (Antenna Toolbox)	
Signal Processing Toolbox™	Functions with <code>gpuArray</code> support (Signal Processing Toolbox)	Code Generation and GPU Support (Signal Processing Toolbox)
Audio Toolbox™	Functions with <code>gpuArray</code> support (Audio Toolbox)	Code Generation and GPU Support (Audio Toolbox)
Wavelet Toolbox™	Functions with <code>gpuArray</code> support (Wavelet Toolbox)	Code Generation and GPU Support (Wavelet Toolbox)
Predictive Maintenance Toolbox™	Functions with <code>gpuArray</code> support (Predictive Maintenance Toolbox)	
Curve Fitting Toolbox™	Functions with <code>gpuArray</code> support (Curve Fitting Toolbox)	
Radar Toolbox	Functions with <code>gpuArray</code> support (Radar Toolbox)	

Plotting Interactively

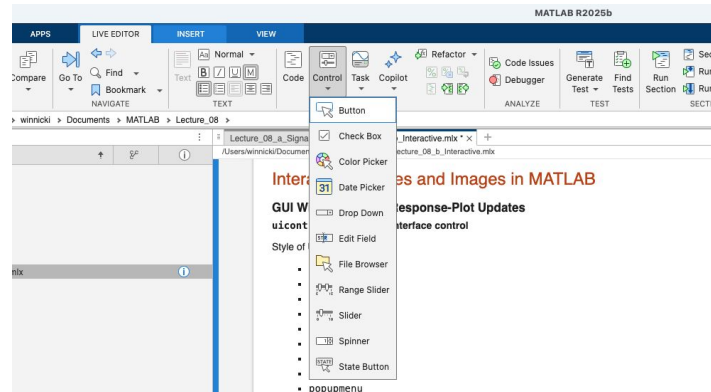


Graphical user interfaces (GUIs), also known as apps, provide point-and-click control of your software applications, eliminating the need for others to learn a language or type commands in order to run the application.

3 ways to create an app in MATLAB

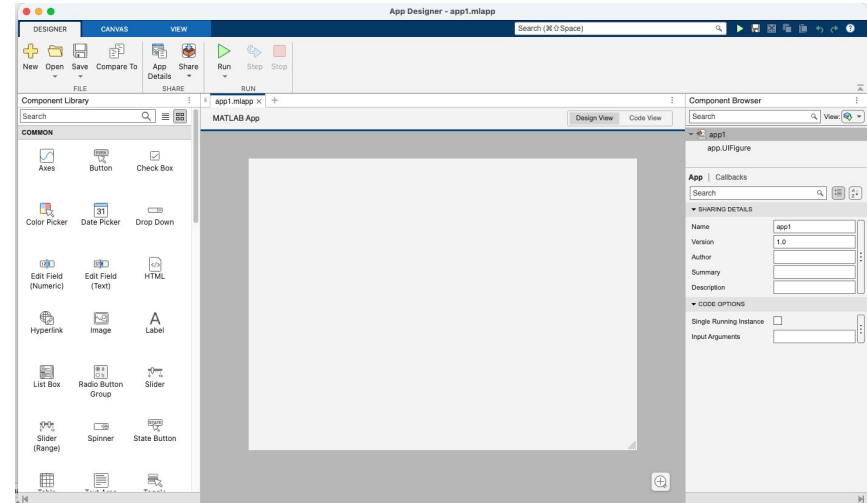
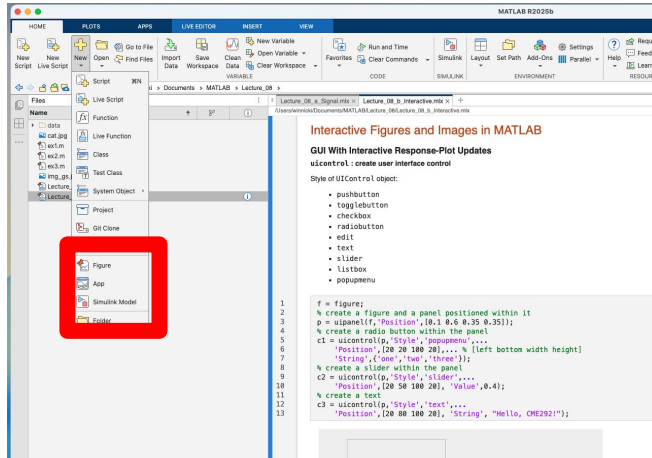
1. Convert a script into a simple app

- Use the Live Editor to convert a script into a simple app that has interactive controls allowing others to experiment with variables in your code.
- Add sliders, dropdowns, edit fields, and buttons without writing any code.
- Specify what parts of the script will run when a value is changed.
- Hide the code to create simple apps and dashboards.



2. Create an app interactively

- Use App Designer, an interactive environment that integrates the two primary tasks of app building
- Lay out the visual components
- Program the app's behavior.



3. Create an app programmatically

- use MATLAB functions to define the layout and behavior of the app

GUI With Interactive Response-Plot Updates

uicontrol : create user interface control

Style of UIControl object:

- pushbutton
- togglebutton
- checkbox
- radiobutton
- edit
- text
- slider
- listbox
- popupmenu

- **uifigure** creates a figure for building a user interface and returns the Figure object.
- **uigridlayout** creates a grid layout manager for an app. It positions UI components along the rows and columns of an invisible grid that spans the entire figure or a container within the figure.
- **uipanel** creates a panel in the current figure and returns the Panel object.
- **uibutton** creates a push button in a new figure and returns the Button object.
- **uislider** creates a slider in a new figure window and returns the Slider object.

Demo

Share Data Among Callbacks

Graphics callback functions must accept at least two input arguments:

- The handle of the object whose callback is executing
 - used within the callback function to refer to the callback object
- The event data structure
 - can be empty for some callbacks or contain specific information that is described in the property description for that object

UserData is a property that all graphics objects in MATLAB possess, and can be used to store any single, user-defined array with a particular object.

Demo

Some interesting examples of GUI on File Exchange

- Interactive Data Navigation Pane Widget

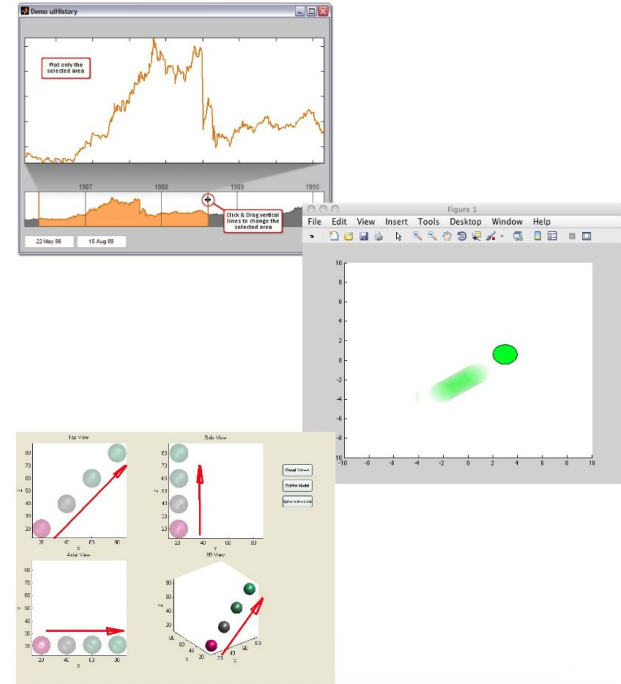
<https://www.mathworks.com/matlabcentral/fileexchange/23423-interactive-data-navigation-pane-widget>

- Moveit2 - Move a graphical object with the mouse

<https://www.mathworks.com/matlabcentral/fileexchange/23304-moveit2-move-a-graphical-object-with-the-mouse>

- Move a 3D object with mouse in a traditional 4-view window

<https://www.mathworks.com/matlabcentral/fileexchange/29140-move-a-3d-object-with-mouse-in-a-traditional-4-view-window>



Identify Coordinates Using Cursor

`[x,y] = ginput(n)`

This allows us to identify the coordinates of n points within Cartesian, polar, or geographic axes.

- To choose a point, move the cursor to the desired location and press either a mouse button or a key on the keyboard.
- Press the **Return** key to stop before all n points are selected.
- The coordinates of your selected points are returned in x,y.

Demo

Create ROI (Region of Interest) in Images

A region of interest (ROI) is a portion of an image that you want to filter or operate on in some way.

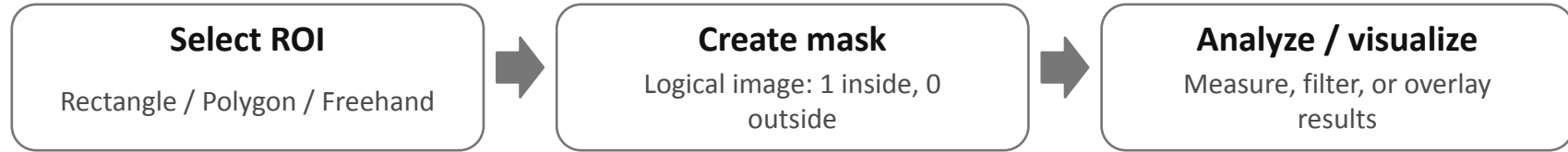
We can represent an ROI as a binary mask image.

In the mask image, pixels that belong to the ROI are set to 1 and pixels outside the ROI are set to 0.

[Image Processing Toolbox]

- masking out region by selection
- freehand ROI

ROI workflow: selection, mask, analysis



- Masking is a representation: it stores membership (inside/outside), not pixel values.
- Once you have a mask, you can apply it, compute statistics, or run morphology/segmentation within the ROI.
- Keep ROI creation separate from analysis so callbacks and GUIs stay simple.

Modern interactive ROI tools (Image Processing Toolbox)

Interactive ROI objects (typical workflow):

```
imshow(I);  
h = drawrectangle; % or drawpolygon, drawfreehand, drawcircle, ...  
mask = createMask(h);  
Iroi = I; Iroi(~mask) = 0;
```

Notes for interactive apps:

- ROI objects have position/shape you can read and update (great for callbacks).
- Use event listeners (e.g., when ROI moves) to trigger recomputation.
- Store ROI handles or masks in app properties / UserData for sharing among callbacks.

Applying a mask to an image

```
% Masking (keep ROI, zero outside):  
Iroi = I; Iroi(~mask) = 0;  
  
% Masking (extract values inside ROI):  
vals = I(mask);  
  
% Masking + overlay visualization:  
imshow(I); hold on;  
h = imshow(cat(3, ones(size(mask)), zeros(size(mask)), zeros(size(mask))));  
set(h, 'AlphaData', 0.25*mask);
```